Cardano Pub/Sub Framework
Design and Architecture

Alexandros Antonov, Evangelos Kolyvas, and Spyros Voulgaris

Department of Informatics
Athens University of Economics and Business
Athens, Greece

Project: Pub/Sub Framework for the Cardano Ecosystem
Deliverable D2 – September 2024

Contents

Chapter 1

Introduction

Publish/Subscribe (Pub/Sub) systems [5, 10] have emerged as a fundamental communication paradigm in distributed systems, enabling scalable and decoupled interactions between data producers (publishers) and consumers (subscribers). In this model, publishers disseminate messages without knowledge of the recipients, while subscribers express interest in specific types of messages, allowing for a loosely-coupled and asynchronous exchange of information. This decoupling of communication is particularly beneficial in large-scale distributed systems, where the number of interacting entities can vary significantly, and network conditions may be unpredictable.

Topic-based Pub/Sub is one of the most widely used variations of Pub/Sub. In a topic-based system, messages are published to topics, and subscribers receive only the messages associated with the topics they have subscribed to. This model provides an intuitive and efficient way to organize communication in applications such as real-time data streaming, distributed databases, event-driven architectures, and Internet of Things (IoT) networks.

Despite its advantages, the design of topic-based Pub/Sub systems poses several challenges. Efficient message dissemination is essential, ensuring that subscribers receive published events with minimal delay while avoiding network congestion. Scalability is critical, particularly in environments with large numbers of topics, large numbers of subscribers per topic, or a high rate of event publication. Ensuring fault tolerance and consistency in the presence of network or node failures remains a key concern in distributed settings. Reliability is of fundamental importance, as no subscriber should miss any event, and no event should get lost once published. Availability of events for a defined retention period beyond their publication is an indispensable feature when reliability is at stake, allowing subscribers that experienced periods of expected or unexpected disconnections to retrieve missed events on demand. Provisioning of resources, by means of either dedicated servers or end-nodes voluntarily contributing their resources, is yet another important parameter in designing a Pub/Sub system. Resource provisioning becomes even more challenging considering the management of a—typically dynamic—membership of nodes, where

publishers and subscribers can join or leave the system frequently, necessitating robust mechanisms for handling such changes without disrupting communication. Last but not least, security mechanisms, including authorization, access control, confidentiality, but also protection against adversary behavior, form an inherent part of Pub/Sub systems' implementation.

This report focuses on the design and architecture of a Pub/Sub system to be used within the Cardano ecosystem. We focus on addressing the core challenges of such a system, emphasizing on the algorithmic aspects of the respective solutions. We leverage a decentralized architecture which blends nicely with the Cardano blockchain, incorporating on-chain mechanisms to enable global knowledge of topics and their administration, ensuring that the system operates efficiently and securely in a dynamic environment.

The remainder of this paper is organized as follows. Section 2 provides an overview of related work in Pub/Sub systems. Section 3 outlines the design of our proposed system, detailing its architecture, membership management, and message dissemination protocols. Section 4 presents the evaluation of the system, demonstrating its scalability, fault tolerance, and performance under varying workloads. Finally, Section 5 discusses the implications of our findings and concludes the paper.

The report is structured along the following three major design points, focusing on efficiency, scalability, reliability, secure administration, and persistent storage, analyzed in the following three chapters, respectively:

- **Chapter 2:** Pub/Sub Administration
- **Chapter 3:** Event Dissemination
- **Chapter 4:** Event Persistence

Finally, we conclude this report in Chapter 5.

Chapter 2

Pub/Sub Administration

Administration in the context of the Cardano Pub/Sub framework involves fundamental actions regarding topic management, including topic creation and deletion, establishing communication constraints between publishers and subscribers, configuring various topic properties, and designating which entities are authorized to perform the aforementioned actions.

2.1 Topic Registry and Properties

Central to our architecture is a global directory of topics, referred to as the *topic registry*, or simply *registry*, storing topic registrations and their respective configurations (as well as *replication servers* that will be discussed in Section 4.1).

The topic registry is implemented by means of a smart contract deployed on the Cardano blockchain. Each topic's registration and properties are stored there, indexed by a unique identifier, known as the *topicId*.

When a node seeks to initialize a topic, it submits a transaction that includes the topic name and metadata specifying the communication constraints. Imposing a transaction fee, for topic initialization ensures a controlled set of available topics, which is essential for the effective implementation of the pub/sub framework.

The owner can later modify the metadata to adjust the communication parameters as the topic evolves. For additional administrative management, the topic owner can delegate these responsibilities to other nodes, called *administrators*. The topic registry performs authorization of nodes attempting to perform certain actions on a topic by confirming that their public key is present in the appropriate list within the topic's metadata.

There is a total of six properties associated with a topic:

1. **TopicId:** This is a 256-bit unique ID assigned by the topic registry (i.e., the smart contract) upon new topic registration. It is guaranteed to be unique for ever, that is, even after the topic's end of life.

2. **Topic name/description:** This is a short string containing an arbitrary name and/or description of this topic. From a technical point of view, this does not need to be unique, however if uniqueness is required, it can be easily imposed by the topic registry.

3. **Owners:** This is a list of one or more *owners* of the topic, namely, the list of their public keys. An account registering a new topic automatically becomes its owner. Any owner may add/remove accounts to/from this list, provided there is always at least one owner left. An owner has the right to modify the topic's configurations, as well as to delete it.

4. **Publishers:** This is the list of accounts, by means of their public keys, who have the right to *publish* on this topic. If this list is empty, any node may publish in this topic without any special permission.

5. **Retention period:** This is the time period for which events of this topic should be retained by the persistence layer (Chapter 4) and should be available on demand.

6. **Replication factor:** This configuration parameter defines how many nodes should store replicas of events published on this topic, for the retention period mentioned above.

   Both the retention period and the replication factor are defined on a per-topic basis.

As explained above, a topic's publishers property can either be empty, or list one or more publishers. We establish that any user may publish a message to a topic if the list of publishers is empty, and in that case we are referring to it as an *open topic*. Once one or more publishers have been added to the list, the ability to publish is restricted exclusively to the nodes included in that list. In that case we are dealing with a *moderated topic*. In moderated topics, nodes involved in event dissemination simply discard events that are not signed by any of the registered publishers.

This is not the case when it comes to subscribers. Our design does not impose any constraints on the subscriber set of any topic. Unlike the case for publishers, implementing such subscription constraints is of questionnable merit, given the dynamic nature of the subscriber group and the ease by which an event being disseminated unencrypted may leak to non-authorized users. Therefore, we define all topics as public, allowing any user to subscribe to any available topic. If privacy is required, encryption should be employed to guarantee event confidentiality, and private channels should be used to manage key distribution.

Regarding the retention period and the replication factor, they are configured on a per-topic basis. This makes more sense from a user point of view for the sake of uniformity within a topic, but it is also mandated by a technical reason. As we will see in Chapter 4, subscribers wishing to retrieve missed events on demand need to have these values independently of the events themselves, namely the events they are currently trying to retrieve.

2.2 Indicative API

The methods corresponding to the administrative actions in our system are detailed in Table 2.1. Notably, each method requires the *topicId* as its first argument, as eligible nodes (either the topic owner or delegated administrators) might manage multiple topics.

The first method, *createTopic*, is used to initialize a topic, providing a name, an initial set of administrators, and an initial set of publishers. It also sets the initial replication factor R and retention period T. The roles of administrators and publishers are discussed in further detail below.

The *deleteTopic* method is used by the topic owners to remove a topic from the list of available pub/sub topics. This operation effectively deletes the topic's data, including its name and metadata, from the pub/sub contract state. The topicId, however, remains in the topic registry for good, to prevent reassignment to any future topic.

The third and fourth methods, *addOwner* and *removeOwner*, are used to add or remove users to the list of owners. Similarly, *addAdmin* and *removeAdmin*, are used to delegate administrative rights to nodes that hold the corresponding private keys. These nodes will have the authority to manage the set of publishers, so they should be chosen carefully by the topic owners.

The following two methods in Table 2.1, *addPublisher* and *removePublisher*, are used to modify the set of nodes authorized to publish events for a specific topic. A participating node checks the origin of a received event before forwarding it during event dissemination. If it is a moderated topic and the event's signature does not match any of the topic's publishers, the message is discarded.

Finally, the last two methods allow and owner or an administrator to configure the topic's replication factor R and retention period T, respectively.

Method	Eligible Node	Description
createTopic(*topicName*, *admins*[ ], *publishers*[ ] *R* /*replication factor*/ *T* /*retention period */) returns (uint256 *topicId*)	owner	Creates a topic, setting its name and an initial set of administrators and publishers, by means of their public keys. Providing an empty *publishers* list will result in an *open topic*. Also sets the initial replication factor R and retention period T (in epochs). Returns the topicId, which is assigned by the topic registry.
deleteTopic(*topicId*)	owner	Deletes the topic identified by *topicId*. Event replicas remain available till the retention period expires. The topicId is kept for good, to guarantee unique ids.
addOwner(*topicId*, *newOwner*)	owner	Adds *newOwner* to the list of topic owners, without removing the existing ones.
removeOwner(*topicId*, *oldOwner*)	owner	Removes *oldOwner* from the list of topic owners, unless this is the only owner.
addAdmin(*topicId*, *newAdmin*)	owner	Adds *newAdmin* to the list of topic administrators, without removing the existing ones.
removeAdmin(*topicId*, *oldAdmin*)	owner	Removes *oldAdmin* from the list of topic administrators.
addPublisher(*topicId*, *newPublisher*)	owner, admin	Adds *newPublisher* to the list of topic publishers, without removing the existing ones.
removePublisher(*topicId*, *oldPublisher*)	owner, admin	Removes *oldPublisher* from the list of topic publisher.
setReplicationFactor(*topicId*, R)	owner, admin	Updates the replication factor to the given value R.
setRetentionPeriod(*topicId*, T)	owner, admin	Updates the retention period to T epochs.

Table 2.1: Administration methods of the Topic Registry smart contract.

Chapter 3

Event Dissemination

Peer-to-Peer (P2P) systems have shown to constitute a remarkably efficient and scalable way for data dissemination at a global scale. In a nutshell, the idea is simple: Each node maintains a small number of neighbors. Upon receiving an unseen event, a node forwards it to all its neighbors, modulo the neighbor it received it from. In the absence of failures and provided that the network overlay, i.e., the graph formed by the nodes and their neighboring relations, is strongly connected, an event is guaranteed to reach all participating nodes. In the face of failures, provided a sufficient number of redundant paths have been set up, the event still makes it to all participating nodes with very high probability.

A pub/sub system encompasses a number of topics, each having its own set of subscribers to which corresponding events should be delivered. Thus, applying the P2P dissemination model in a pub/sub setting entails establishing one individual dissemination overlay per topic, as well as a global network structure combining all these overlays. This global network structure should allow a newly joining node that contacts any arbitrary peer in it to easily and reliably discover the individual overlay(s) of the specific topic(s) it wishes to join, and should let it participate in event dissemination therein.

Effectively, this calls for a two-faceted approach in organizing the overall network topology, namely, building a composite network structure that allows:

(i) fast, efficient, and reliable event dissemination *within a topic*,

(ii) quick navigation to the desired topic's subscribers *across topics*.

Sections 3.1 and 3.2 present the design and network topologies satisfying the two aforementioned goals, respectively. Then, Section 3.3 presents the protocols used to build and maintain these topologies.

3.1 Dissemination within a Topic

Large-scale data dissemination has two fundamental goals: *reliability* and *efficiency*. A number of metrics assessing the quality of a dissemination mechanism can be categorized around these goals, as listed below.

(A) **Reliability metrics:** These are the metrics concerning the reliability of message delivery to all participating nodes. More specifically:

- *Hit ratio*: This is the fraction of nodes that receive a message over the total population. In the absence of failures and node churn it should be 100%, while in the face of failures and churn it should still be as close to that as possible.
- *Resilience to failures and churn*: For a dissemination system to be meaningful in a real-world dynamic network, it should operate reasonably well in the presence of node or link failures, and node churn. The operation under such conditions is evaluated by means of the hit ratio, described above.

(B) **Efficiency metrics:** These are the metrics concerning the efficiency of dissemination, in terms of dissemination speed and resources used. More specifically:

- *Dissemination speed*: The time required for the dissemination of a particular message to complete. The faster a message is disseminated the better. Dissemination speed depends on two principal factors. First, the hop-to-hop delay in forwarding messages (processing delay on nodes plus network propagation delay). Second, the number of hops a message takes to reach the last node.
- *Message overhead*: The overall number of times a message is forwarded during its dissemination. For a message to reach N recipients, it should be forwarded a minimum of N times. In practice, however, messages are forwarded a number of redundant additional times, to sustain churn and failures. Message overhead rates a dissemination system with respect to preserving or wasting network resources.
- *Load distribution*: The distribution of load over nodes, in terms of messages received and messages forwarded. Ideally, load should be evenly distributed among participating nodes.

In order to satisfy both reliability and efficiency requirements, we adopt the dissemination algorithm proposed in Hybrid Dissemination [18]. Hybrid Dissemination decouples these two fundamental goals by combining a number of random links among nodes for efficiency with a Harary graph topology for reliability, as explained below.

3.1.1 Efficiency

In Hybrid Dissemination, each node maintains a small number of f links to random other nodes subscribed to the same topic. When a node generates a new message or receives one it has not previously seen, it forwards it to all its f neighbors, without sending it back to the neighbor it received it from. Value f is a system-wide parameter, called the *fanout*.

This algorithm is very efficient at spreading messages to a considerable percentage of the nodes very fast, specifically at exponential speed with base f: A new message progressively reaches f^0 (=1, the message generator), f^1, f^2, ... other nodes. Consequently, a message spreads very fast even for small values of $f \geq 2$. As expected, dissemination slows down when the message is forwarded to nodes that have already received it. However, if the selection of nodes to forward a message to is uniformly random, this slowdown is expected to be negligible in the first stages of dissemination and until the message has reached a substantial percentage of the network.

Despite this algorithm's strength at spreading messages fast, it does not guarantee that every single node will receive the message being disseminated, even in the absence of failures. This is because a directed random graph is not necessarily strongly connected. That is, random links alone do not provide the required dissemination reliability.

3.1.2 Reliability

In order to make message dissemination reliable, Hybrid Dissemination also utilizes a Harary graph structure.

A *Harary graph* [7] of n nodes and connectivity t, denoted $H_{t,n}$, is a minimal link graph that is guaranteed to remain connected when up to $t-1$ nodes or links fail. That is, it exhibits a minimal cut of t. Notably, in Harary graphs links are equally distributed across all nodes, each node maintaining exactly t bidirectional links to its neighbors.

Visualizing a Harary graph is easy. We need to consider an arbitrary cyclic ordering of nodes, and establish t bidirectional links on each node, to its $t/2$ closest neighbors in each direction (assuming an even t for our needs, although the formal Harary graph definition covers odd values too). Figure 3.1 illustrates three Harary graphs of 9 nodes each, with connectivities 2, 4, and 6, respectively.

The Harary graph links provide reliability. They guarantee connectivity of all nodes in a single strongly connected component. Therefore, when a message is being disseminated, it will reach all nodes. However, dissemination over the Harary graph alone is slow, as a message has to traverse the entire topology sequentially.

3.1.3 Efficiency and Reliability: Putting it all together

To guarantee both efficiency and reliability in event dissemination, we combine both random links and the Harary graph structure.

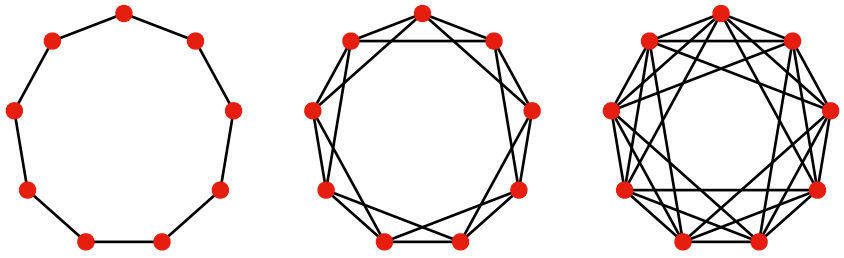


Figure 3.1: Harary graphs of 9 nodes and connectivities 2, 4, and 6, respectively.

In its most lightweight version, we employ a Harary graph of connectivity two, which essentially is a ring topology, and we also establish just one random link per node. This means that when a node receives a message from one of its two ring links, it forwards it to its other ring link as well as to its random link. When a node receives a message through its random link, it forwards it along both ring links. This uses a very low fanout of two, while still achieving fast dissemination in a logarithmic number of steps and providing a high degree of resilience to failures, as shown in [18].

Figure 3.2 illustrates the dissemination of a message in the face of failures. The ring structure is intentionally broken into a number of disconnected components. Intuitively, the use of random links delivers a message to each component with very high probability, while the use of ring links guarantees that a message will traverse all nodes of a component, up to the very last one.

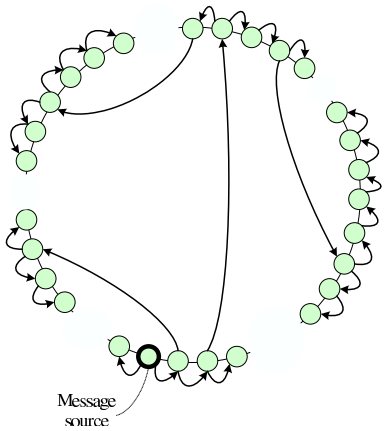


Figure 3.2: Example of a message dissemination in a partitioned ring. For clarity, only a few of the followed random links are shown.

3.2 Navigation across Topics

The navigation layer helps nodes navigate through the overlay to reach nodes of the same topic in a timely manner.

In our model, a new node may join by contacting any arbitrary node of any topic. Subsequently, it should be able to quickly and reliably navigate to the topic(s) it wishes to subscribe to, that is, to find other subscribers of its topic(s) of interest. The same applies to nodes that are already established subscribers in some topic(s), but wish to subscribe to additional topics too.

To achieve that, we augment our global overlay structure with a *navigation layer*. Rather than letting nodes come across same-topic peers relying purely on random encounters, which could take unpredictably long given the possibly high number of topics, the navigation layer defines a circular proximity metric by which nodes progressively get *closer* to their same-topic peers, and eventually connect to them.

In this layer, every node is expected to maintain a small number of links to nodes of *other topics*, in order to be able to point newly joining nodes towards their target. These links should follow a structured logic, to safely route joining nodes to their targets in a small number of steps.

We utilize the fact that in the Cardano Pub/Sub System all topics are registered on-chain and, thus, are globally known. This allows us to define a global cyclic ordering of topics, essential for routing new nodes to their targets. In doing so, we consider a mapping of all T topics on an enumeration from 0 to $T-1$, as illustrated in Figure 3.5.

In its simplest form, the navigation layer could just demand from each node subscribed to a topic with enumeration x to maintain at least one link to some arbitrary subscriber of topic $(x+1)\ mod\ T$, that is, to a random subscriber of the successor of topic x in this global cyclic ordering.

To make routing efficient, we additionally demand from each node to maintain links to topics at distances $b^0 = 1$, b^1, b^2, ..., b^i, in both clockwise and anticlockwise directions, with i being as high as possible without b^i exceeding the total number of topics divided by two, $T/2$. Parameter b is called the *routing base*, and has a typical value of 2. Figure 3.6 shows an example for a network of ten topics with routing base $b = 2$. These links are akin to the links known as *fingers* in Distributed Hash Tables (DHTs), discussed in Section 4.2.1.

When a node p is directing a node q towards q's target topic, it checks all its finger links, and it provides it with the one whose enumeration is numerically as close as possible to that of the target topic. If all finger links are in place, node q is guaranteed to reach subscribers of its target topic in $\Omega(\log_b T)$ hops in the worst case.

3.3 Building the Event Dissemination Overlay

The last two sections presented the structure of the overlays required for disseminating events within a topic (Section 3.1), and for navigating to any ar-

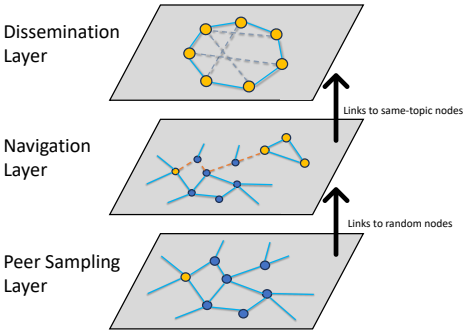


Figure 3.3: Three-layer protocol. Each layer constitutes a distinct gossiping protocol, maintaining its own list of neighbors and gossiping with them. Lower layers help higher layers by equipping them with links from their own neighbors.

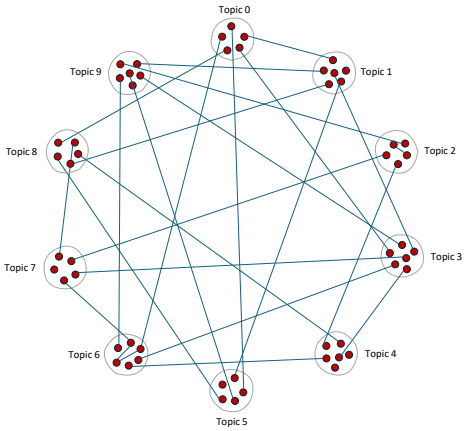


Figure 3.4: **Peer Sampling Layer:** Random graph to maintain the network connected, and to provide a source of random samples.

bitrary topic's overlay (Section 3.2). This section presents fully decentralized P2P algorithms that let nodes self-organize into an overlay combining both aforementioned structures.

Building the overlay for event dissemination is a three-faceted process. We address it by deploying three distinct gossiping protocols, depicted as three layers in Figure 3.3. The layers and their roles are as follows:

1. **Peer Sampling Layer:** First, all subscribers, irrespectively of the topics they are subscribed to, need to form and maintain a single, universal, connected overlay, preventing them from getting seggregated into disjoint components, even in the face of failures. This layer performs random peer sampling for discovering peers, which it subsequently provides to the navigation layer directly above it, as illustrated in Figure 3.3. For this, we employ *SecureCyclon* [1], a secure and efficient instance of the *peer sampling* family of protocols [8].

2. **Navigation Layer:** Second, subscribers should be able to efficiently navigate and discover other same-topic subscribers, as explained in Section 3.2. This layer introduces a notion of proximity among topics, based on their global ordering. It establishes denser links between nodes of nearby topics, while links become sparser between nodes of more distant topics. For that, we use *Vicinity* [17], a fully decentralized overlay building protocol.

3. **Dissemination Layer:** Finally, subscribers of the same topic should be able to disseminate events among themselves fast and reliably. As explained in Section 3.1, this can be achieved by having nodes form a Hybrid Dissemination topology, combining a Harary graph with a number of ran-

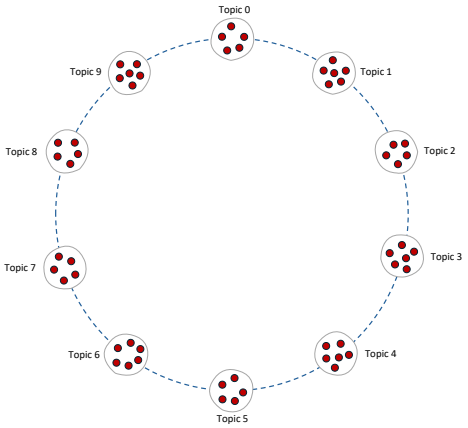


Figure 3.5: Considering topics in circular ordering, harnessing the global knowledge of topics.

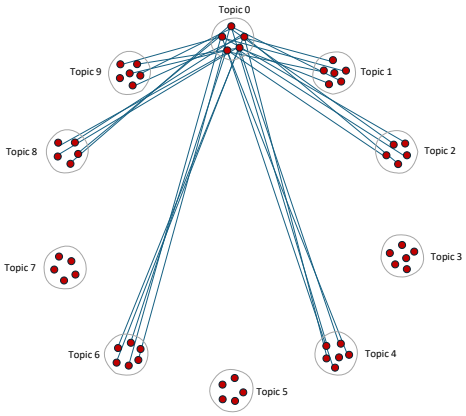


Figure 3.6: **Navigation Layer:** Each subscriber maintains a link to some node at topic-distance 2^i, for $i = 0, 1, 2, \ldots$

dom links. To achieve this, we will again employ the Vicinity protocol, tailored to enable nodes to create and maintain the Hybrid Dissemination overlay.

Each node maintains, separately per layer, a list of links to other nodes, its *neighbors* for that layer, collectively known as the node's *view* of the network. On behalf of each layer, each node periodically performs *gossip*, that is, it contacts one of its neighbors in that layer's view and they exchange a few links in an effort to improve their views. The three layers are closely intertwined and operate in a cooperative manner, assisting each other in improving their views and reaching their goals faster.

The following sections detail the aforementioned layers, discussing their rationales and reasoning on their design choices. We present them in a bottom-up order.

3.3.1 Peer Sampling Layer

Peer Sampling protocols [8] form a well-studied family of gossiping protocols known for maintaining robust P2P overlays in a self-organizing manner and with strong self-healing properties.

The Peer Sampling layer serves two primary functions. First, it guarantees that the entire set of nodes remains connected in a single connected component. Second, it constitutes a continuous source of random peer samples, that is, fresh links to randomly chosen peers among all alive nodes in the network. The continuous link provision ensures the systematic replacement of old—and possibly obsolete—links by newly acquired, fresh links. This is key to peer discovery.

We have opted to use SecureCyclon [1], a secure version of the Cyclon protocol [16], which is a popular representative of the peer-sampling protocol family. SecureCyclon was developed at the Athens University of Economics and Business (AUEB) in the context of the *"Eclipse-Resistant Network Overlays for Fast Data Dissemination"* project funded by IOG (then IOHK) in 2020-2022.

Cyclon is highly scalable and lightweight in network resources, as each node is only required to retain a very small, fixed-sized view and to engage in small gossip exchanges once per cycle, irrespectively of the network size.

The SecureCyclon variant of the protocol enforces nodes to strictly follow the gossip-exchange rules, preventing them from deviating from them in ways that could alter the protocol's properties. Specifically, it mandates that every node in the overlay periodically generates only a *limited* number of fresh links, as dictated by the protocol. This mechanism prevents malicious nodes from manipulating the connections to and from legitimate nodes and from flooding the network with links to themselves, inhibiting the formation of hubs by adversaries and ensuring they cannot dominate legitimate node communications.

As shown in [16], Cyclon exhibits the following robustness properties, which are consequently inherited by SecureCyclon too:

- Extreme robustness to node failures: The overlay graph remains connected even when a high proportion of overlay nodes concurrently lose connectivity.
- Self-Healing: The overlay graph adapts to node failures, maintaining the structural characteristics of a random graph.
- Randomness: Each node's neighbors are *uniformly randomly* selected among all alive nodes in the network. Periodically, a node's neighbors are dynamically updated by some of them being replaced by other randomly selected neighbors. Thus, this protocol provides a continuous source of truly random peer samples.

These properties render SecureCyclon a highly suitable substrate layer for our protocol, offering a reliable groundwork for the construction of the other two layers.

Figure 3.4 illustrates the overlay, with nodes grouped per topic, showing the random links established by the peer sampling layer.

3.3.2 Navigation Layer

The navigation layer aims at building the topology described in Section 3.2 and illustrated as an example in Figure 3.6 for ten topics.

This layer employs Vicinity [17], a gossiping protocol that lets nodes self-organize from an arbitrarily connected initial overlay into a structured topology. Following the classic gossiping paradigm, each node maintains a view of k neighbors and periodically selects a random one of them to initiate a gossip exchange with. Then, the two nodes send each other a few links (i.e., a few of their neighbors), which they accommodate to update their views.

The target structure is defined by means of a *selection function*, which, provided a reference node p and a set of k or more candidate neighbors for p, selects the k best picks to bring the overlay as close as possible to the target topology. In a gossip exchange this function is used both to select the links to *send* to the gossip partner, as well as to select the links to *keep*, among the ones received from the gossip partner. Assuming a transitive proximity property of the selection function, in the sense that neighbors selected by node p in prior gossip exchanges are likely to help p further improve its view if gossiped with, all nodes keep improving their views, helping the topology converge fast to the target structure.

As explained in Section 3.2, the navigation layer models all T topics as vertices in a ring structure, enumerated from 0 to $T-1$ (Figure 3.5). The proximity between two topics is defined as the Euclidean distance between them in this ring, in modulo arithmetic. A node's goal is to pick neighbors belonging to topics at distances b^0, b^1, b^2, ... (with b being the *routing base* with a typical value of 2) in both clockwise and anticlockwise directions, as well as to its own topic. In line with Chord terminology, we will be referring to these topics as *finger topics* of a given node. Thus, each node has $2 \cdot \left(\lfloor log_b \frac{T}{2} \rfloor + 1\right) + 1$ finger topics, that is, $\lfloor log_b \frac{T}{2} \rfloor + 1$ on each side, plus its own topic.

The selection function retains, for each finger topic t, the c nodes that are subscribers of topics as close to t as possible, ideally of t itself. Among nodes of different topics, the ones closer to the respective finger topic are preferred. Among, however, nodes of the same topic, the selection function favors fresher links over older ones, to help in recycling neighbors and pushing departed nodes out of the system in a natural way. With each node sharing the same objective, the probability of a neighbor close to finger topic t being able to provide links to nodes even closer to or belonging to t, gradually increases. Thus, initially the selection function helps nodes establish links to peers in their finger topics, while once this has been achieved, it helps nodes refresh their links and replace them by newer ones, to the same finger topics.

When a node p is selecting which links to send to its gossiping counterpart q, it feeds its entire view into the selection function, setting q as the reference node, and setting the number of links k equal to the number of finger topics. In plain words, node p goes through q's finger topics, and for each one it selects the closest peer from its entire view, and ships all these links to q. That is, it picks the peers that would be most useful for q.

When node p receives a set of links from a gossiping counterpart, it feeds its existing links (i.e., its current view) along with its newly received links to this selection function, which selects the neighbors to be retained and to form its updated view. This time, the selection function is instructed to keep c links per finger topic of node p.

Let us walk through a realistic scenario to demostrate the layer's scalability with the number of topics. A typical value for the routing base b is $b = 2$, while c typically has a small value, between 1 and 5. Assume, first, that we have a total of 1001 topics. Each node has exactly 500 topics on each side, which

translates into a total of 19 finger topics (9 in each direction, at distances $2^0 = 1$ to $2^8 = 256$, plus its own topic). For a moderate value of $c = 3$, this would account to maintaining a total of 57 neighbors per node, which constitutes a negligible load. In case of a ten-fold increase in the number of topics, i.e., for a total for 10,000 topics, the total number of finger topics per node increases to 27, and the neighbors per node to 81.

3.3.3 Dissemination Layer

This layer is responsible for creating the topology described in Section 3.1, as illustrated in Figure 3.7. At a fully converged state, each node should maintain one or more random links to other subscribers of the same topic, along with two *ring links*: one to its predecessor and one to its successor in the ring. While the random links used by this layer are provided directly by the navigation layer (discussed in the previous section), this layer is specifically responsible for establishing and maintaining the ring links.

To achieve this, we once again employ the Vicinity [17] protocol, but with a modified selection function. Initially, a topic overlay consists of nodes that belong to the same topic, but are connected through random links. Each node starts with neighbors that have arbitrary IDs, but gradually refines its view by acquiring nodes with the closest IDs, from the view of its gossiping partners. This process ultimately results in a ring structure where all same-topic nodes are ordered according to their IDs.

The Vicinity selection function employed for this purpose relies on a single criterion, namely, node IDs. For any given node, gossiping is periodically initiated within the dissemination layer by selecting the neighbor whose ID is

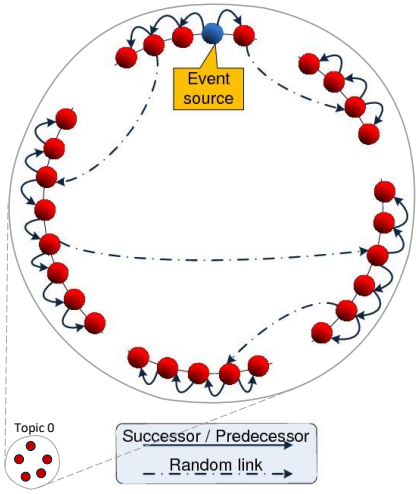


Figure 3.7: **Dissemination Layer:** Subscribers within a topic form a Hybrid Dissemination overlay.

closest to that of the node. During gossip, the function selects the k candidates whose IDs are closest to that of the reference node. Specifically, it selects $k/2$ candidates with the closest lower IDs and $k/2$ candidates with the closest higher IDs, in modulo arithmetic to preserve continuity.

When node p selects neighbors to send to q, it will not only consider its dissemination layer view but also the same-topic neighbors in its navigation view, which contains random nodes of the same topic. This exposes node q to a random sample of same-topic nodes, increasing the likelihood that some will be of close ID proximity. This significantly accelerates convergence, as each gossip exchange exposes gossip partners to each other's randomness. With each node sharing the same objective, the probability of a node establishing connections to its successor and predecessor steadily increases with the progression of gossiping, as it gradually gets connected with neighbors closer to its ID.

Chapter 4

Event Persistence

The dissemination mechanism described in the previous chapter ensures efficient message delivery to all alive subscribers of the respective topic. This section addresses the reliability aspect, focusing on subscribers who are offline when a new event is published, and offers an alternative method for them to retrieve the event.

Nodes that happen to be offline during the publishing and dissemination of an event, e.g., because they were experiencing a transient network failure, suffered a system crash, or were simply turned off, should be able to retrieve the missed event(s) on demand later on. That is, the events should be retained by the system for some period. Interested subscribers should be able to identify which events they have missed during their absense, and to retrieve them explicitly on demand.

Due to the potentially huge number of published events, interested subscribers should be able to efficiently locate and access explicitly the specific events they have missed. Effectively, the pub/sub system should provide a *persistence layer*, storing published events and ensuring they remain available on demand for a certain duration following their publication. This duration is known as the *retention period* (Table 4.1).

The nature of this event storage system calls for the use of a Distributed Hash Table (DHT). DHTs are decentralized systems comprising a—typically large—number of individual nodes that collectively provide a storage service, allowing the *storing* and *lookup* of data records. For storing, data has to be submitted in the form of ⟨*key*,*value*⟩ tuples. For lookups, the requester has to provide a *key*, and the DHT retrieves the corresponding *value*. Each record is stored on one or more nodes, determined by the DHT based on the record's key. The number of nodes maintaining a copy of a given record is known as the record's *replication factor* (Table 4.1), and effects the record's availability in the face of failures.

Three main questions arise:

1. **Resource Provisioning:** How will nodes for providing resources for

Parameter	Description
Retention period	The amount of time an event should be available for, since it was published.
Replication factor	The number of replica servers that should keep a copy of an event during its retention period.

Table 4.1: Key parameters of the persistence layer

event storage be selected?

2. **Key-Value Store:** Which DHT algorithm will be employed to implement the key-value store.
3. **Indexing Scheme:** Which key-scheme will be used for indexing events in the key-value store.

These questions are addressed in the following three sections, respectively.

4.1 Resource Provisioning

To successfully implement replication services, two key issues need to be addressed:

1. Appointing Replication Servers: The first challenge lies in appointing nodes to act as replication servers. This task is not trivial, as replication servers must commit to their responsibilities by remaining available and providing the necessary storage resources. A server that frequently leaves and rejoins the system will lead to costly reconfigurations and data migration.
2. Incentivization and Penalization Mechanisms: The second issue involves designing appropriate incentives and penalties for replication servers. These mechanisms are essential to motivate replication servers to contribute their resources consistently and to ensure that they honor their commitments.

In the context of blockchains, where nodes typically act out of self-interest, there is a challenge in encouraging them to allocate network, memory, and computational resources for the benefit of others. However, if service facilitation is critical to the system's functionality, we must devise strategies to ensure participation.

4.1.1 Appointing Replication Servers

Trust in replication servers plays a crucial role in ensuring the reliability of the persistence layer within the Cardano Pub/Sub system. To maintain this trust,

replication servers should be associated with Cardano accounts, and in particular with accounts that have an established interest in enhancing the system's performance and overall experience, such as *Stake Pool Operators* (*SPOs*).

We require each replication server to be registered on-chain, with its IP address and the public key of its associated Cardano account recorded, alongside a binding commitment to provide services for a defined period, measured in epochs. This registration process ensures accountability and transparency in service provisioning. The *topic registry* described in Section 2.1 could be used for that, or a separate smart contract.

To further strengthen this trust, the entity operating the replication server should be required to meet certain eligibility criteria, such as holding a minimum asset threshold in ADA. This requirement helps ensure that only well-resourced and reliable entities are allowed to participate, reducing the risk of service disruptions.

Replication servers can leave the system, but to prevent unexpected behavior and to reduce node churn, joining and leaving can be scheduled only at epoch boundaries.

4.1.2 Incentivization and Penalization

Incentivization and penalization mechanisms are essential to ensure that replication servers provide consistent and reliable services. The workflow of each server must be monitored to determine whether it successfully stores the data it was assigned to hold. Based on the result, the server is either rewarded or penalized financially.

To this end, we propose that interested entities (such as SPOs), who register replication servers on-chain, commit a security deposit (in ADA) from their account, much like the existing staking mechanism. This security deposit will be refunded once the replication server is unregistered, provided it has fulfilled its service commitment for the entire period.

The account should receive periodic monetary rewards for the service provided. These rewards will be funded by the event publishers, who will pay according to the replication factor and the retention period they choose for their events. However, if a replication server fails to meet its storage obligations, a monetary penalty will be applied by withholding part or all of the security deposit associated with the server. Nodes that exhaust their deposits through penalties will be disqualified from participating in event storage, and removed from the set of active replication servers in the blockchain.

An incentivization mechanism as the one described above implies a reliable method of periodically challenging the replication servers to verify whether they are indeed storing copies of their assigned events. A number of protocols, such as those from the *Proof of Replication* [2, 6] and *Proof of Retrieval* [3, 13, 9] families, have been suggested. However, while feasible on commodity hardware, these protocols may be prohibitively expensive to execute directly on-chain, in a smart contract context.

The incentivization mechanism for replication servers is a separate line of research, which is work in progress, and will not be discussed further in this document.

4.2 Key-Value Store

As discussed earlier, the specific requirements of the Cardano Pub/Sub persistence layer regarding efficiency and scalability and the inherently distributed nature of the Cardano ecosystem, call for Distributed Hash Tables (DHTs) as the recommended implementation of the needed key-value store.

4.2.1 Background on DHTs

Distributed Hash Tables (DHTs) are Peer-to-Peer (P2P) systems that allow the *storage* and efficient *lookup* of data records across a large set of distributed nodes. DHTs are fully decentralized, in the sense that no central coordinator is required. Instead, the storage/lookup service is collectively offered by all nodes together. Moreover, nodes are *symmetric*, in the sense that no node has any special role or extra responsibilities.

Key Mapping

In a nutshell, DHTs operate as follows. Nodes acquire unique IDs uniformly distributed in a very large space $\mathbb{S}$, where typically $\mathbb{S} = [0, 2^{256})$. Data records are in the form of $\langle key, value \rangle$ tuples, with unique keys. Keys are assumed to belong to the same space $\mathbb{S}$ and to be also uniformly distributed in it, otherwise their hashes with values in $\mathbb{S}$ are considered instead. When storing a data record, its key is used to unambiguously appoint the authoritative node that will be responsible to store a copy of it. Typically, this is the node with the arithmetically closest ID to the key, or with an ID that is just lower (or just higher) than the key, in modulo arithmetic. When performing a lookup, the provided key is used in the same way, to reach that same node and to query whether such a tuple has been stored (and retrieve its value) or not.

Data Replication

To improve the availability and reliability guarantees in the face of failures, in practice data records are replicated on more than just a single node. The number of replicas kept for a data record is known as its *replication factor*, R. A data record with replication factor R is, thus, copied on R distinct nodes. These nodes could be defined as the R nodes with IDs arithmetically closest to the key. Alternatively, they could be defined relative to the authoritative node described above, e.g., by including that node and the $R-1$ ones with just higher (or lower) IDs, or by splitting these $R-1$ nodes in half on both sides of the authoritative node.

Load Balancing

Regarding the storage load, given our assumption of a uniform distribution for both node IDs and keys, statistically it is equally split across all participating nodes.

Routing

An essential part of any DHT is its *ID-based routing algorithm*, which, given a target key and starting at any arbitrary node, enables the navigation to the corresponding node(s) responsible for that key. A number of different DHTs exist, such as Chord [15], Pastry [12], Kademlia [11], each employing a different routing algorithm. The basis for all these DHTs is the aforementioned circular 256-bit ID space, out of which nodes are assigned unique, uniformly distributed IDs.

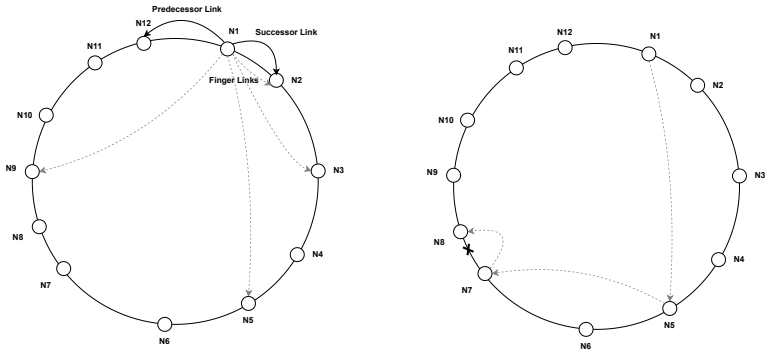
DHTs are designed with scalability in mind, aiming to scale to millions or even billions of nodes. However, maintaining direct knowledge of such a vast number of nodes and managing continuous dynamic updates in a real-world network of that size, is impractical.

To address this challenge, DHTs use a common approach to scale their routing algorithms. The idea is to define some notion of proximity, and to require nodes to maintain accurate knowledge of their close neighborhood, but only sparse knowledge of the rest of the network. That is, a node has to maintain links to every single node in its close proximity, while it gradually keeps fewer links to nodes at increasingly longer distances. This way, long-range links to distant IDs can be followed during the initial routing steps, to quickly get a query closer to the target ID's proximity, while gradually more accurate short-range links can be used to locate the actual target node.

In Chord, for example, each node knows its direct successor and predecessor in ID space , while it also knows nodes at ID distances 2^i, for $i = 1, 2, \ldots$, known as *finger links*(Figure 4.1a). A store/lookup request for a given key should be handled by the first node whose ID is greater than or equal to the key, in modulo arithmetic. When presented with a key, a node forwards the request clockwise as far as it can, without, however, crossing the key (Figure 4.1b). A node noticing that its direct successor has an ID equal to or higher than the key, forwards the request to its successor, informing it that it is the authoritative node responsible for that key, and the routing process completes. Note that the ID distance to the target is roughly halved in each routing step, consequently reaching the target in $O(\log_2 n)$ hops, where n is the number of nodes in the DHT.

Maintenance

Building and maintaining an ID-based routing infrastructure for a DHT is not trivial. A DHT first needs to establish the appropriate links between nodes and continuously monitor for network changes—such as nodes joining, leaving, or failing—to dynamically adapt and repair the routing overlay, using self-healing



(a) The link structure of a node in a Chord network.

(b) Navigation in a Chord network.

Figure 4.1: Abstraction of the Chord DHT. Each node in the ring maintains links to its successor, predecessor, and distant nodes (finger links). The routing process leverages these links to efficiently forward key requests, reducing the distance to the target in logarithmic steps.

techniques. Overlay maintenance algorithms are specific to the network structure of each individual DHT, and are out of the scope of this report.

Last but not least, a DHT is also in charge of dynamically migrating replicas to reflect changes taking place in the network. When a node joins or departs, the assignment of data records replicas in its ID vicinity changes. When a node joins, it takes over responsibility for some replicas from other nodes. Likewise, when a node departs, other nodes in its ID vicinity become responsible for the records it was keeping, based on the new replica allocation. In the former case (join), it is always possible to take over responsibility and copy the respective replicas from other nodes. In the latter (departure), assuming the worst case of sudden, non-graceful departures due to failures, data records with a replication factor higher than the number of simultaneously failed node are guaranteed to be preserved, by copying their replicas to healthy nodes in close ID proximity.

Again, the mechanisms for replica maintenance are specific to the individual overlay topology of each DHT. However, the general idea is that nodes periodically ping their nearby nodes in ID space to check on their liveness, and when a node is found to be non-responsive it is considered dead, and other nodes (the next successors / predecessors in line) have to make copies of the departed node's data records, to keep their replication factor fixed.

4.2.2 DHT for the Cardano Key-Value Store

Given the discussion above and the specific requirements for the Cardano key-value store, we present the structure and operation of the proposed DHT.

Overlay Structure and Routing

Two key observations can be made regarding the specific requirements of the persistence layer's storage system. First, the number of replication servers is anticipated to be relatively limited, comparable to or smaller than the number of Stake Pool Operator (SPO) relays, likely in the range of hundreds to thousands. Second, these replication servers will be registered on-chain and committed for a defined number of epochs (Section 4.1.1). Effectively we are dealing with a system that provides *global knowledge of membership*, which comprises a relatively *small* and *stable* set of replication servers.

We take into account the aforementioned observations to adopt an efficient and robust *one-hop* DHT. This DHT is based on a clique structure. That is, every node knows the IDs and IP addresses of all the others. Evidently, any node may locally identify and directly contact—in a single hop—the node(s) responsible for storing data associated with any arbitrary key.

This design bares strong similarities to Amazon's *Dynamo* [4] key-value store, a production-level system that powers hundreds of Amazon services since 2007, and which has lent its features to Amazon's *DynamoDB* [14], a distributed NoSQL database.

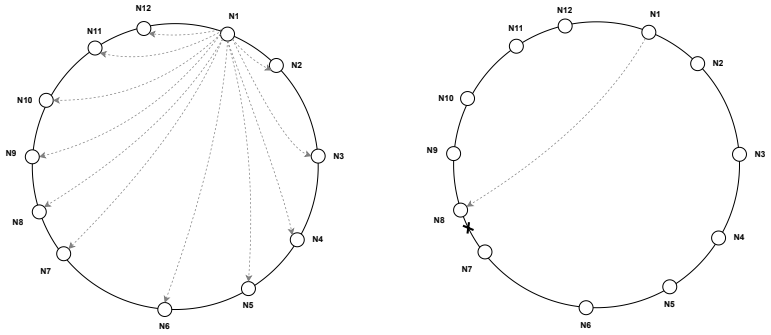
This design has a number of advantages:

- **Simplified Overlay Management**: Since every node knows all other nodes, handling joins, departures, and failure recovery is greatly simplified, removing the need for a complex routing infrastructure.
- **Increased Performance**: Nodes can access the target responsible for any key in a single hop, leading to significantly faster lookup times and reduced network overhead.
- **Reduced Routing Overhead**: Nodes responsible for storing data are not burdened with routing overhead, allowing them to focus solely on their storage and retrieval duties.
- **Enhanced Fault Tolerance**: Faults can be detected quickly, allowing for faster replication of data on alternative nodes, ensuring higher availability and reliability of stored events.

Figure 4.2a illustrates the links of a single node within a Cardano DHT topology. When compared to traditional DHT overlays like Chord, as shown in Figure 4.1a, our network appears denser, establishing a significantly higher number of links. Typically, building and maintaining such clique-like overlays is challenging, as each node should continuously verify that its links point to alive nodes. This approach is particularly difficult to achieve in environments with a large number of nodes and high churn rates.

However, our design makes this approach feasible due to the following characteristics:

- *Global IP knowledge*: In typical network overlays, establishing connections with other nodes usually requires peer discovery mechanisms, such



(a) The link structure of a node in the Cardano DHT network.

(b) One-hop navigation in the Cardano DHT network.

Figure 4.2: Abstraction of the Cardano DHT. Each node in the ring maintains links to all other nodes in the overlay, forming a clique overlay. The routing process leverages these links to instantaneously forward key requests, minimizing routing time and network overhead.

as random walks or gossiping protocols. In our case, replication servers can immediately initiate link establishment procedures since they already know the network addresses of all others via the blockchain.

- *Simplified link maintenance*: Constantly pinging all nodes in the overlay to verify their activity and discarding inactive nodes would create significant network overhead. In our design, this process is streamlined. Each node is strongly incentivized to stay active for its committed duration to avoid penalties. Nodes that remain inactive for an extended period are automatically removed from the set of replication servers on the blockchain, which is visible to all active servers. Active nodes continuously monitor the blockchain for updates to the list of replication servers and adjust their connections accordingly.

- *Limited node count*: Maintaining millions of active links simultaneously is not feasible due to resource constraints. However, in our case, the expected number of replication servers is limited to a few hundred or thousand nodes, making this approach practical.

As observed in Figure 4.2b, routing in our approach is conducted in one single hop, which results in improved lookup times in comparison to Figure 4.1b, consistent lookup delays, and reduced network overhead.

Data Replication

To ensure effective event replication, we enable the configuration of the event replication factor R on a per-topic basis. The R parameter for a topic can be set either during the topic's initialization or modified at any time by invoking the pub/sub smart contract, similar to how administrative parameters are adjusted as described in Chapter 2.

When a node publishes an event e on topic t, it signs the event, it computes the corresponding key k following the indexing scheme described in Section 4.3, and hands the $\langle k,e \rangle$ tuple to any arbitrary replication server, along with t's topic ID. The event tuple is, then, routed directly to the R_t replication servers with closest IDs to key k, leveraging the DHT's one-hop routing.

Similarly, when a lookup request for an event of topic t is made, the requesting node computes the key k and passes the request to any arbitrary replication server, along with t's topic ID. That replication server, taking topic t's replication factor R_t into account, knows which replication servers should be storing copies of that event. It picks and contacts one of them at random, to distribute the load among them, and the actual event's content is sent back to the requesting node.

Maintenance

To maintain an active set of nodes replicating each event, the following strategy is employed. Assume a replication server p and let E_p be the set of events currently stored by p. Let S_e be the set of replication servers responsible to store a copy of event e. Node p computes its *monitoring set* M_p as the union of servers that are responsible for any of the events that p has. That is, $M_p = \bigcup_{e \in E_p} S_e$. These are the replication servers whose health p should be keeping an eye on. Thus, p periodically pings nodes in its monitoring set M_p, for example by sending one ping per second to a different node each time, visiting all of them in a round-robin fashion.

If a node q is found to be unavailable, p will collect the affected set of events it knows of, that is, all events in E_p for which q was responsible for too, it will compute the set of servers responsible for these events in the absense of q, and will inform them all about q's unavailability. This step is to ensure that all servers that should take action on behalf of q's failure do get informed the soonest possible.

At the same time, as soon as a replication server (including p) gets informed about q's unavailability, it checks for itself which events stored at q it should take over, and fetches them directly from other replicas of these events.

This recovery mechanism is only triggered when a node is confirmed to be absent from the DHT overlay. To ensure accuracy, a node must make several consecutive ping attempts with short intervals and timeouts. Specifically, three ping attempts with a timeout of 5 seconds per attempt, is recommended. This results into a total potential downtime of 15 seconds before a node is deemed inactive, providing a balance between responsiveness and avoiding false positives

from temporary network issues. These parameters can be fine-tuned based on simulations reflecting the real-world conditions of the Cardano SPO ecosystem.

4.3 Indexing Scheme

Having an efficient and robust key-value store is the cornerstone of a well functioning persistence layer. However, an educated planning of which information should be indexed by which key, is no less crucial for the correct, efficient, and truly decentralized operation of such a system.

More specifically, it is crucial to devise an indexing scheme that satisfies the following two conditions:

1. It balances the storage and lookup load across all replication servers evenly.
2. It lets subscribers construct the keys of missed events on their own, without depending on any central component.

To illustrate the importance of these design decisions, let us see why a naive approach failing to satisfy any of these conditions could prove inapt for our use-case.

Naive approach 1: Assume that all events published on topic T were grouped by the same key (e.g., *hash(T)*) and mapped on the same replication servers. In terms of correctness, there would be no problem. Subscribers would know where to get missed events on topic T from, therefore condition (2) would be met. All information would be readily available for them. However, these replication servers would effectively be forced to operate as central repositories for *all* events of topic T, possibly among other topics mapped on them too. This would impose a disproportionally high load on them in comparison to other nodes, resulting in uneven load balancing, poor service, and a non-scalable system, failing to satisfy condition (1).

Naive approach 2: Let us now imagine an approach where each individual event was mapped on a different set of replication servers, indexed by a unique key (e.g., the hash of the event's content and its timestamp concatenated). This would satisfy condition (1), distributing events evenly across all replication servers. However, a registry would be needed to list the keys of events published per topic, to enable subscribers to perform the respective lookups. Such a registry would need to store centrally some information for each single published event, failing condition (2) and negating the benefits of decentralization, and, effectively, harming the system's performance and scalability.

Recommended approach: In order to satisfy both aforementioned conditions, we propose events to be indexed by the following key scheme:

$$key = hash\big(\ \langle \textsc{topic} \rangle\ .\ \langle \textsc{publisher} \rangle\ .\ \langle \textsc{sequence nr} \rangle\ \big)$$

where dots denote concatenation. Parameter TOPIC is the topic's unique name or ID, while PUBLISHER is the publisher's public key, which is also unique.

The SEQUENCE NR is a per-topic, per-publisher counter, maintained by each publisher for each topic it publishes in. It starts at zero and increases by one for each new event a node publishes on that topic.

This scheme satisfies condition (1), as it indexes each event independently of any other, spreading them evenly across all replication servers. It also satisfies condition (2), as subscribers may easily construct the keys of missed events on their own, provided they know which publishers have published on a given topic, and remembering the last sequence number they have received. Then, they may simply keep increasing each publisher's sequence number, until they reach a number for which no event has been published.

There are still two problems regarding the local generation of keys by subscribers. First, if a new publisher starts publishing on a topic while a subscriber has been offline, the subscriber is not aware of this publisher to start trying out its sequence numbers. Second, for topics with a large number of publishers, subscribers would have to try out a large number of possible event keys.

To alleviate the last two problems, we do require a truly lightweight registry per topic. This registry will be logging, for each publisher, only the sequence number of its *last* published event and a corresponding *timestamp*. Such a log should be indexed by the topic's hash alone, to make it readily addressable by subscribers. Therefore, a single replication server (or group of them) would be responsible for a given topic, which introduces some degree of centralization. However, the amount of information stored is negligible, as topics are expected to have a small number of publishers, and should only be updated once per event published on that topic. Even in the unlikely case of topics with hundreds or thousands of publishers, this number would still be finite and manageable, unlike the ever-growing number of events that would need to be individually logged in one of the naive key schemes portrayed as counter examples earlier.

The workflow is as follows. When a subscriber of topic T recovers from a period being offline, it contacts the key-value store with $hash(T)$ for key, providing the respective DHT node with the timestamp of when it went offline. That node responds with the list of publishers (i.e., their public keys) who published events on this topic since that moment in time, along with their latest respective sequence numbers. At this point, the subscriber knows exactly which ranges of sequence numbers it has missed per publisher, so it produces the corresponding keys and spawns requests on the key-value store to retrieve these events.

In regards to performance, note that these requests can be shipped concurrently, to let the respective queries execute in parallel. Also, given the topic's replication factor R, a random one of the R replication servers keeping replicas of an event, or of the topic log in the initial step, can be contacted, to further spread the load across many different nodes.

Chapter 5

Conclusions

This report has explored the design of a decentralized pub/sub message exchange system tailored for the Cardano ecosystem. The system is structured around three main components, each playing a critical role in ensuring efficient, reliable, and scalable communication between publishers and subscribers.

At the core of this system lie the users, who require a mechanism to seamlessly exchange messages. The proposed design employs a three-faceted protocol that begins by forming a well-connected random graph as the foundational layer. From there, nodes navigate the ID space through arbitrary links, ensuring they eventually connect with nodes that belong to the same topic group they want to send or receive messages. Finally, at the top layer, nodes periodically gossip with each other, creating a Harary-like graph with some additional random links. This structure ensures both reliability and efficiency in communications, where messages are delivered to the intended nodes with high probability, quickly, and with low overhead, while maintaining balanced load distribution.

To address resilience, the system incorporates an event persistence component, primarily supported by dedicated nodes, the replication servers. These nodes, typically operated by SPOs, are incentivized through rewards to store data for a certain period, ensuring that subscribers can retrieve missed events after either expected or unexpected disconnections. The system leverages a clique-structured DHT, enabling rapid event retrieval through one-hop routing, and highly reducing complexity and overhead.

A crucial component of the system is the topic registry, a smart contract that transparently manages the state of various pub/sub components. Specifically, the topic registry holds information about topics and about replication servers. It implements an authorization mechanism for enabling the controlled administration of topics on a multitude of configuration options, from their creation and deletion, to the management of publishing permissions and event persistence options, essential features for maintaining system integrity. Furthermore, the smart contract governs replication servers, facilitating the formation of the clique topology.

Bibliography

[1] Alexandros Antonov and Spyros Voulgaris. SecureCyclon: Dependable Peer Sampling. In *2nd IEEE International Conference on Distributed Computing Systems (ICDCS 2023)*, pages 1–12. IEEE, 2023.

[2] Juan Benet, David Dalrymple, and Nicola Greco. Proof of replication. *Protocol Labs, July*, 27:20, 2017.

[3] Kevin D Bowers, Ari Juels, and Alina Oprea. Proofs of retrievability: Theory and implementation. In *Proceedings of the 2009 ACM workshop on Cloud computing security*, pages 43–54, 2009.

[4] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's Highly Available Key-Value Store. In *Proceedings of Twenty-First ACM SIGOPS Symposium on Operating Systems Principles*, SOSP '07, page 205–220, New York, NY, USA, 2007. Association for Computing Machinery.

[5] Patrick Th. Eugster, Pascal Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. The Many Faces of Publish/Subscribe. *ACM Comput. Surv.*, 35(2):114–131, 2003.

[6] Ben Fisch, Joseph Bonneau, Nicola Greco, and Juan Benet. Scaling proof-of-replication for filecoin mining. *Report. Protocol Labs Research*, 2018.

[7] Frank Harary. The maximum connectivity of a graph. In *Proceedings of the National Academy of Sciences*, volume 48, pages 1142–1146, 1962.

[8] Márk Jelasity, Spyros Voulgaris, Rachid Guerraoui, Anne-Marie Kermarrec, and Maarten van Steen. Gossip-based peer sampling. *ACM Transactions on Computer Systems*, 25(3):8–es, 2007.

[9] Ari Juels and Burton S Kaliski Jr. Pors: Proofs of retrievability for large files. In *Proceedings of the 14th ACM conference on Computer and communications security*, pages 584–597, 2007.

[10] Anne-Marie Kermarrec and Peter Triantafillou. XL Peer-to-Peer Pub/Sub Systems. *ACM Computing Surveys (CSUR)*, 46(2):1–45, 2013.

[11] Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the xor metric. In *IPTPS '01: Revised Papers from the First International Workshop on Peer-to-Peer Systems*, pages 53–65, London, UK, 2002. Springer-Verlag.

[12] Antony Rowstron and Peter Druschel. Pastry: Scalable, decentralized object location and routing for large-scale peer-to-peer systems. In *IFIP/ACM Middleware 2001*, Heidelberg, Germany, November 2001.

[13] Hovav Shacham and Brent Waters. Compact proofs of retrievability. *Journal of cryptology*, 26(3):442–483, 2013.

[14] Swaminathan Sivasubramanian. Amazon DynamoDB: A Seamlessly Scalable non-Relational Database Service. In K. Selçuk Candan, Yi Chen, Richard T. Snodgrass, Luis Gravano, and Ariel Fuxman, editors, *SIGMOD Conference*, pages 729–730. ACM, 2012.

[15] Ion Stoica, Robert Morris, David Liben-Nowell, David R. Karger, M. Frans Kaashoek, Frank Dabek, and Hari Balakrishnan. Chord: A Scalable Peer-to-peer Lookup Protocol for Internet Applications. *ACM/IEEE Trans. Netw.*, 11(1):17–32, February 2003.

[16] Spyros Voulgaris, Daniela Gavidia, and Maarten van Steen. CYCLON: Inexpensive Membership Management for Unstructured P2P Overlays. *Journal of Network and Systems Management*, 13(2):197–217, June 2005.

[17] Spyros Voulgaris and Maarten Van Steen. Vicinity: A pinch of randomness brings out the structure. In *Middleware 2013: ACM/IFIP/USENIX 14th International Middleware Conference, Beijing, China, December 9-13, 2013, Proceedings 14*, pages 21–40. Springer, 2013.

[18] Spyros Voulgaris and Maarten van Steen. Hybrid dissemination: Adding determinism to probabilistic multicasting in large-scale p2p systems. In *Middleware*, pages 389–409, 2007.